INDEX

Sandhiya.E
Graphics and Multimedia

NAME: Sandhiya.E STD.: _____ SEC.: _____ ROLL NO.: _____ SUB.: _____

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.a	21.7.2026	Bresenham Line Drawing Algorithm		
1.b.	21.7.2026	Bresenham Midpoint Circle Drawing Algorithm		
1.c.	28.7.2026	Bresenham Ellipse Drawing Algorithm		
2.	4.8.2026	Implementation of DDA Algorithm		
3.	4.8.2026	Build an Object using Bresenham Line Algorithm		
4.	11.8.26	Draw a Traffic Signal		
5.	11.8	Olympic logo		
6.	25.8	Butterfly using ellipse		
7.	25.8	Smiling face using ellipse		
8.	8/9	Implement scan conversion of polygon & use flood fill		
9.	15/9	Implement 2D Transformation		

Scanned with CamScanner

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
10.	29.10.26	Polygon Clipping		
11.	29.10.26	3D Transformation		
12.	6.10.26	Create and render 3D objects		
13.	6.10.26	Create a multimedia application		
14.	6.10.26	Create a program that captures video / audio from a webcam		
15	6.10.26	Create an application that allows to send and receive multimedia messages.		
16.	6.10.26	Create a simple 3D model using Blender		

Completed

Scanned with CamScanner

RAJALAKSHMI
ENGINEERING COLLEGE
An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai

BONAFIDE CERTIFICATE

NAME SANDHIYA E

ACADEMIC YEAR 2026-27 SEMESTER V BRANCH CSD

UNIVERSITY REGISTER No. 2116241701048

Certified that this is the bonafide record of work done by the above student in the

IT23E31 - GRAPHICS AND MULTIMEDIA Laboratory during the year 2026 - 2027

16/10/26

Signature of Faculty - in - Charge

Submitted for the Practical Examination held on .....

Internal Examiner External Examiner

Scanned with CamScanner

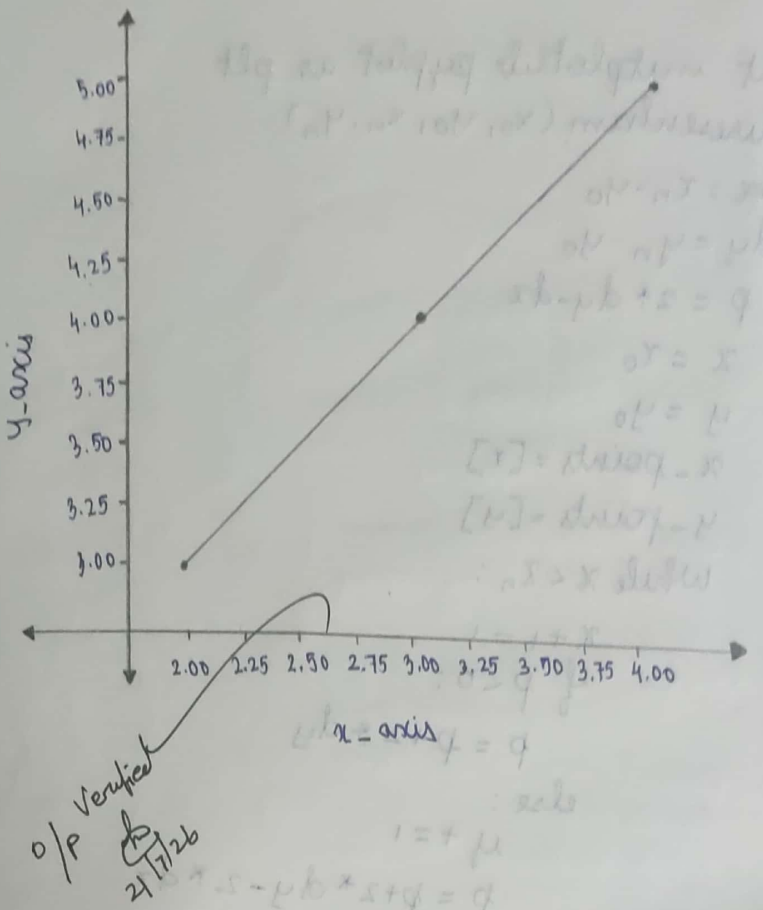
Bresenham Algorithm

Aim: To write a python program to plot a line using bresenham algorithm

```
import matplotlib.pyplot as plt
def bresenham(x0, y0, xn, yn):
    dx = xn - y0
    dy = yn - y0
    p = 2*dy - dx
    x = x0
    y = y0
    x_points = [x]
    y_points = [y]
    while x < xn:
        x+1 = 1
        if p<0:
            p = p+2*dy
        else:
            y += 1
            p = p+2*dy-2*dx
        x_points.append(x)
        y_points.append(y)
    return x_points, y_points
x0 = int(input("Enter x0: "))
y0 = int(input("Enter y0: "))
xn = int(input("Enter xn: "))
yn = int(input("Enter yn: "))
x_points, y_points = bresenham(x0, y0, xn, yn)
print("Points on the lines: ")
for(x, y in zip(x_points, y_points):
    print(f"({x},{y}")
```

Scanned with CamScanner

Bresenham Line Drawing Algorithm (|m|<1)



O/P Verified
21/7/26

Scanned with CamScanner

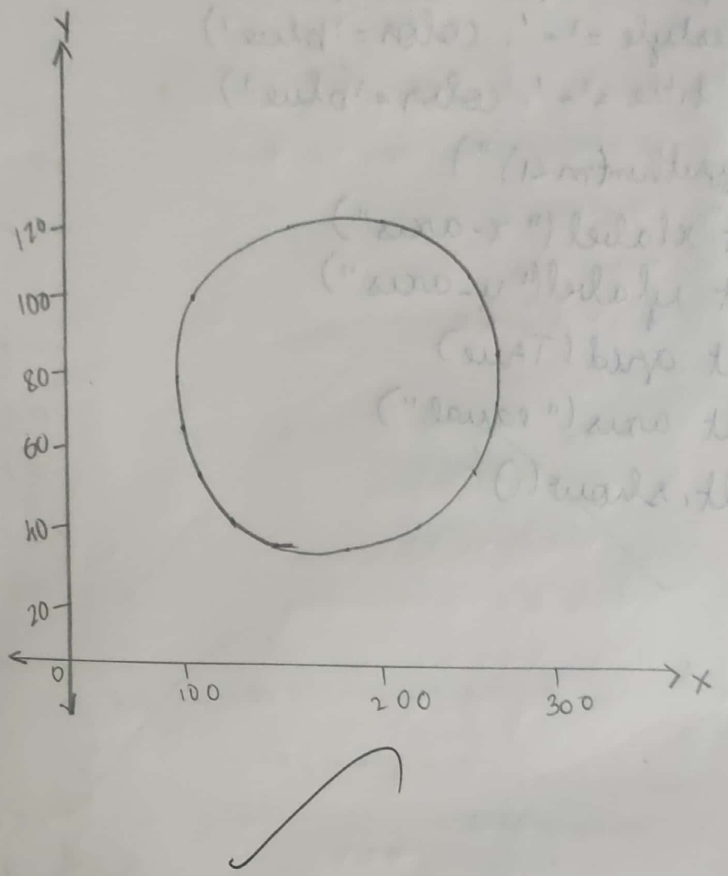
```
plt.figure(figsize=(6,6))
plt.plot(x_points, y_points, marker='s',
linestyle='-', color='blue')
plt.title='-', color='blue')
Algorithm(m<1)")
plt.xlabel("x-axis")
plt.ylabel("y-axis")
plt.grid(True)
plt.axis("equal")
plt.show()
```

Result:

Therefore a straight line has been plotted using Bresenham's Algorithm.

Scanned with CamScanner

Output:



Scanned with CamScanner

Bresenham Midpoint Circle Drawing Algorithm

Aim:

To write a python program to draw a midpoint circle using Bresenham's Algorithm

Program:

```
import matplotlib.pyplot as plt
r = int(input("Enter radius: "))
x = 0
y = r
p = 1-r
while x<=y:
    plt.scatter([x,y,-x,-y,-x,-y,x,y],
    [y,x,y,x,-y,-x,-y,-x], color="blue")
    x+=1
    if p<0:
        p = p+2*x+1
    else:
        y-=1
        p = p+2*(x-y)+1
plt.axis("equal")
plt.grid()
plt.show()
```

Result:

Therefore a midpoint po circle has been drawn successfully using Bresenham's algorithm.

Scanned with CamScanner

28/7

Bresenham Ellipse Drawing Algorithm

Aim:

To write a python program to draw an ellipse using Bresenham's Algorithm

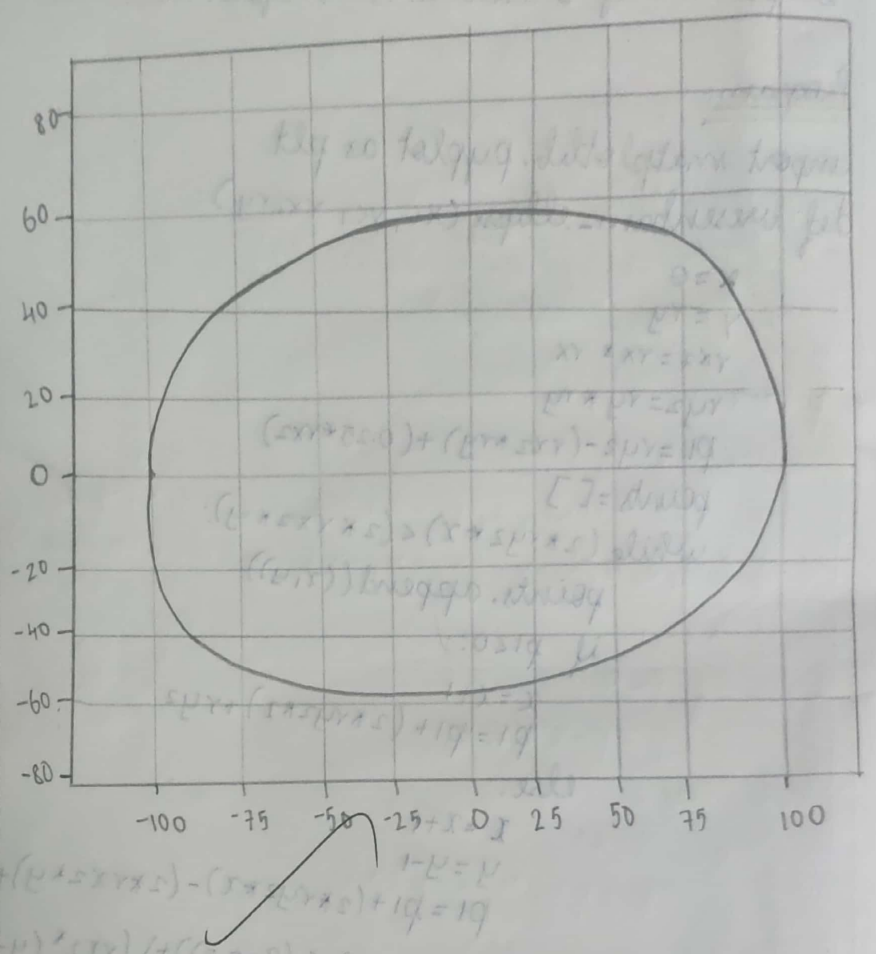
Program:

```
import matplotlib.pyplot as plt
def bresenham_ellipse(xc, yc, rx, ry):
    x = 0
    y = ry
    rx2 = rx * rx
    ry2 = ry * ry
    p1 = ry2 - (rx2 * ry) + (0.25 * rx2)
    points = []
    while (2 * ry2 * x) < (2 * rx2 * y):
        points.append((x, y))
        if p1 < 0:
            x = x + 1
            p1 = p1 + (2 * ry2 * x) + ry2
        else:
            x = x + 1
            y = y - 1
            p1 = p1 + (2 * ry2 * x) - (2 * rx2 * y) + ry2
    p2 = (ry2 * (x + 0.5) * (x + 0.5)) + (rx2 * (y - 1) * (y - 1))
        - (rx2 * ry2)
    while y >= 0:
        points.append((x, y))
        if p2 > 0:
            y = y - 1
            p2 = p2 - (2 * rx2 * y) + rx2
        else:
            x = x + 1
            y = y - 1
            p2 = p2 + (2 * ry2 * x) - (2 * rx2 * y)
                + rx2
```

Scanned with CamScanner

Output:

Bresenham's Ellipse Drawing Algorithm



Scanned with CamScanner

```
    ellipse_points = []
    for x, y in points:
        ellipse_points.append((xc + x, yc + y))
        ellipse_points.append((xc - x, yc + y))
        ellipse_points.append((xc + x, yc - y))
        ellipse_points.append((xc - x, yc - y))
    return ellipse_points

xc = 0
yc = 0
rx = 100
ry = 50
points = bresenham_ellipse(xc, yc, rx, ry)
x_values = [p[0] for p in points]
y_values = [p[1] for p in points]
plt.scatter(x_values, y_values, s=2)
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Bresenham's Ellipse Drawing Algorithm")
plt.axis("Equal")
plt.grid()
plt.show()
```

4/8/25

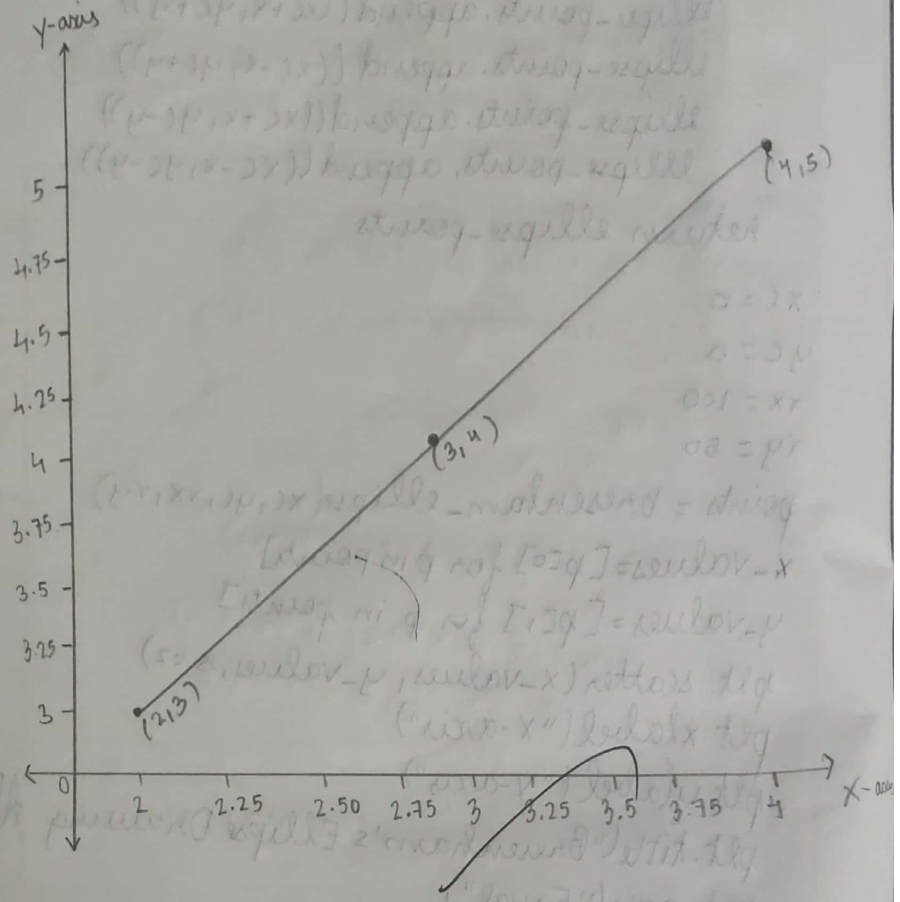
Result:

Therefore an ellipse has been drawn successfully.

Scanned with CamScanner

Output:

DDA Line Drawing Algorithm



Scanned with CamScanner

4.8.2016

Implementation of DDA Algorithm

Aim:

To write a python code to implement DDA Algorithm.

Program:

```
import matplotlib.pyplot as plt
x1 = int(input("Enter x1: "))
y1 = int(input("Enter y1: "))
x2 = int(input("Enter x2: "))
y2 = int(input("Enter y2: "))
dx = x2-x1
dy = y2-y1
steps = max(abs(dx), abs(dy))
x_inc = dx/steps
y_inc = dy/steps
x = x1
y = y1
x_points = []
y_points = []
for i in range(steps+1):
    x_points.append(round(x))
    y_points.append(round(y))
    x += x_inc
    y += y_inc
plt.plot(x_points, y_points, marker='o')
plt.title("DDA Line Drawing Algorithm")
plt.xlabel("x-axis")
plt.ylabel("y-axis")
plt.grid(True)
plt.show()
```

4/8/16

Result:

Therefore, DDA algorithm has been implemented successfully.

Scanned with CamScanner

Build an Object using Bresenham's line Drawing Algorithm

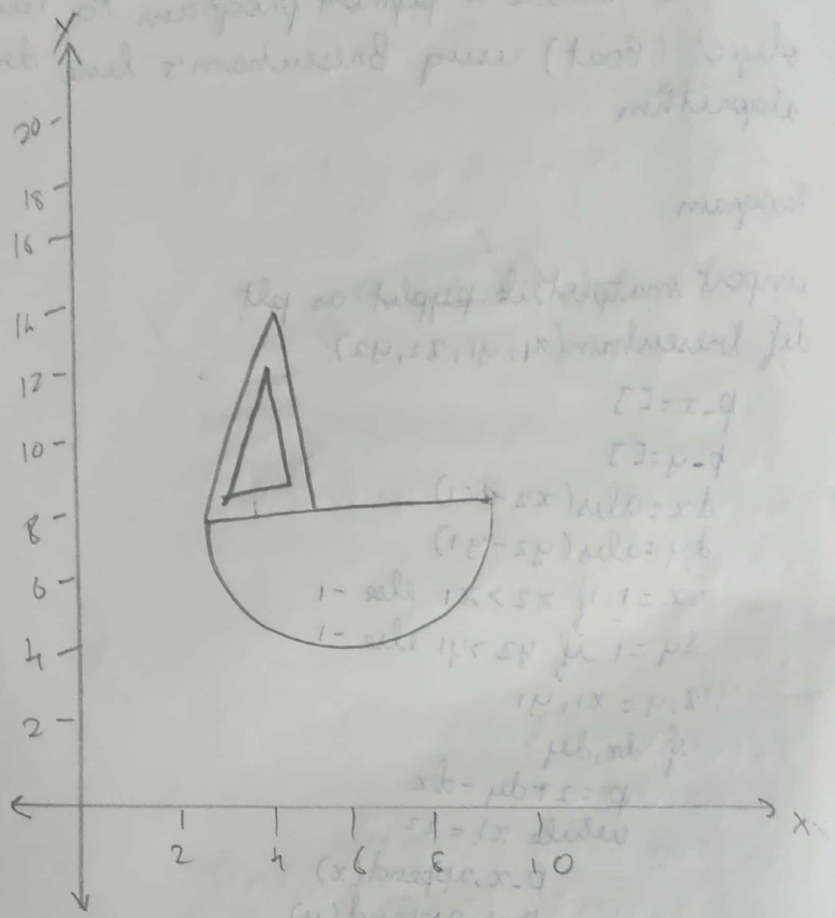
Aim:

To write a python program to build an object (Boat) using Bresenham's line drawing algorithm.

Program:

```
import matplotlib.pyplot as plt
def bresenham(x1, y1, x2, y2):
    p_x=[]
    p_y=[]
    dx=abs(x2-x1)
    dy=abs(y2-y1)
    sx=1 if x2>x1 else -1
    sy=1 if y2>y1 else -1
    x,y=x1,y1
    if dx,dy:
        p=2*dy-dx
        while x1==x2:
            p_x.append(x)
            p_y.append(y)
            if p>=0:
                y+=sy
                p+=2*(dy-dx)
            else:
                p+=2*dy
            x+=sx
    else:
        p=2*dx-dy
        while y1==y2:
            p_x.append(x)
            p_y.append(y)
            if p>=0:
                x+=sx
                p+=2*(dx-dy)
```

Scanned with CamScanner



Scanned with CamScanner

```
        else:
            p+=2*dx
        y+=sy
    p_x.append(x2)
    p_y.append(y2)
plt.plot(p_x,p_y,'k-',linewidth=2)
plt.fill([0,100,100,0],[5,5,80,80], color="lightskyblue")
plt.fill([0,100,100,0],[0,0,5,5], color="deepskyblue")
plt.fill([20,30,70,80],[20,10,10,20], color:"saddlebrown")
plt.fill([50,50,70],[20,50,60], color="white")
bresenham(20,20,80,20)
bresenham(20,20,30,10)
bresenham(30,10,70,10)
bresenham(70,10,80,20)
bresenham(50,20,50,60)
bresenham(50,60,70,40)
bresenham(70,40,50,20)
bresenham(50,60,58,55)
bresenham(58,55,50,50)
bresenham(0,5,100,5)
plt.title("Boat using Bresenham Line Algorithm")
plt.xlim(0,100)
plt.ylim(0,80)
plt.gca().set_aspect('equal')
plt.grid(True)
plt.show()
```

Result:

Therefore a boat has been drawn successfully.

Scanned with CamScanner

4/8

Draw a Rectangle

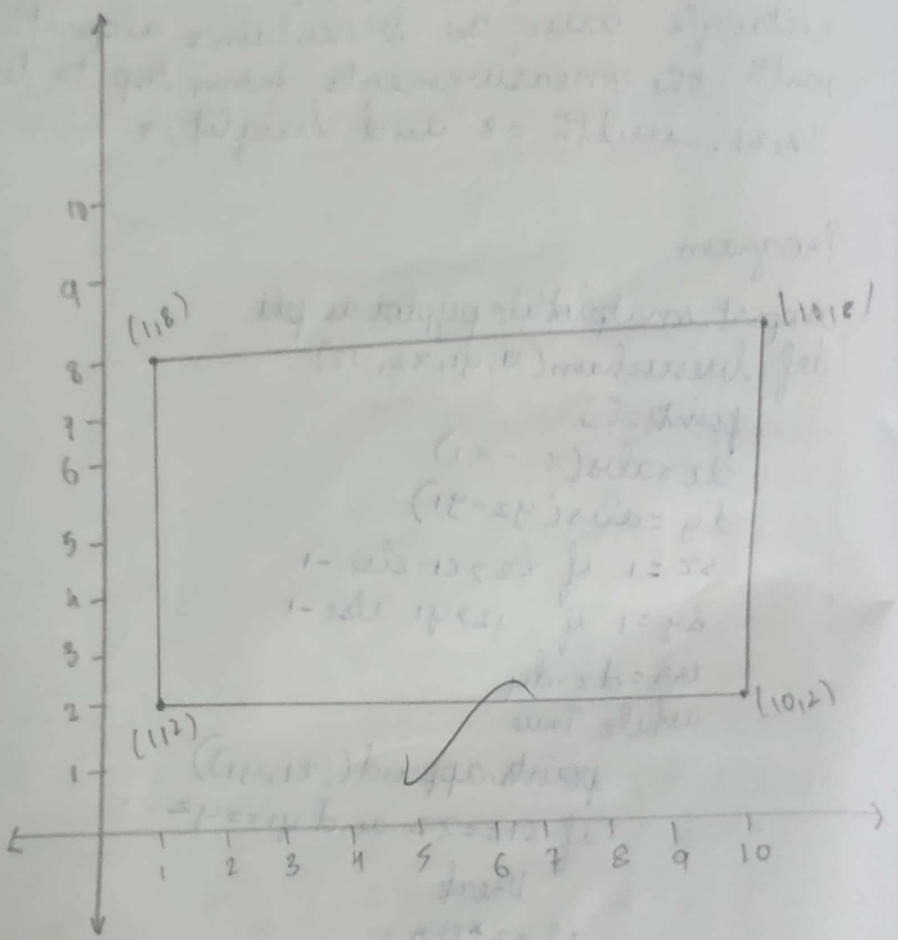
Aim:

To write a python program to draw a rectangle using the bresenham's algorithm with the measurements from top to left (2,2), width = 8 and height = 8

Program:

```
import matplotlib.pyplot as plt
def bresenham(x1,y1,x2,y2):
    points=[]
    dx=abs(x2-x1)
    dy=abs(y2-y1)
    sx=1 if x2>x1 else -1
    sy=1 if y2>y1 else -1
    err=dx-dy
    while True:
        points.append((x1,y1))
        if x1==x2 and y1==y2:
            break
        e2=2*err
        if e2>-dy:
            err-=dy
            x1+=sx
        if e2<dx:
            err+=dx
            y1+=sy
    return points
x1,y1=2,2
x2,y2=10,2
x3,y3=10,7
x4,y4=2,7
```

Scanned with CamScanner



Scanned with CamScanner

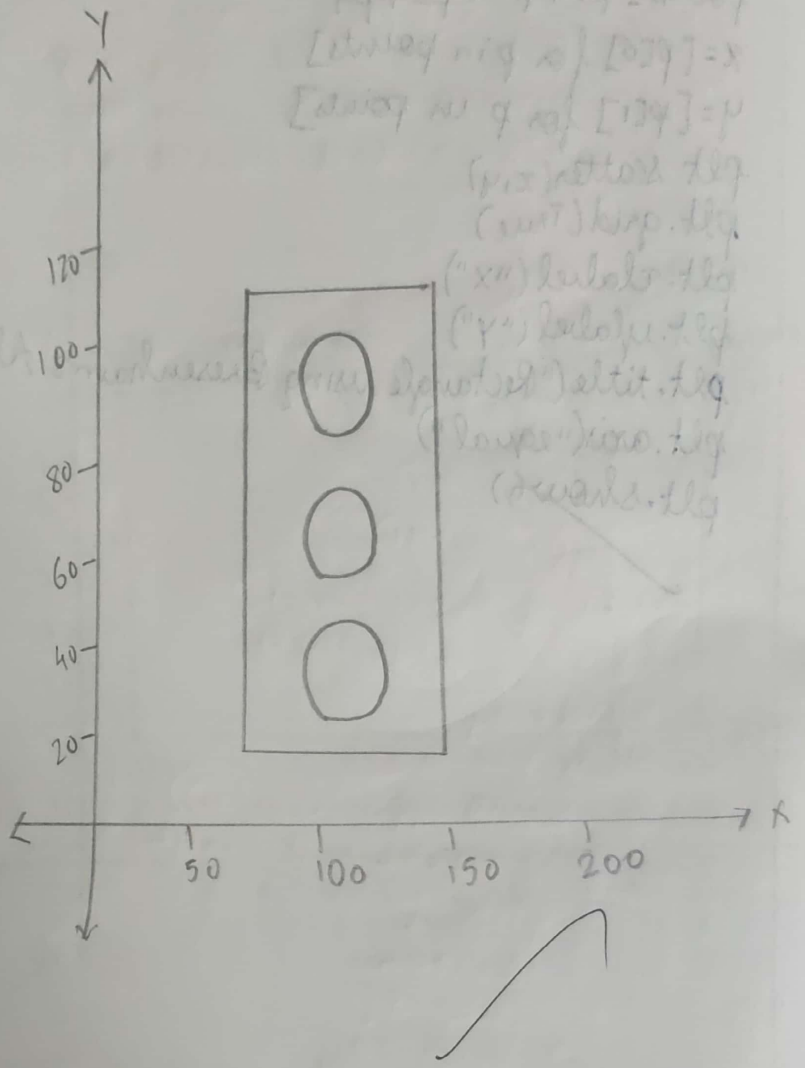
```
p1 = bresenham(x1, y1, x2, y2)
p2 = bresenham(x2, y2, x3, y3)
p3 = bresenham(x3, y3, x4, y4)
p4 = bresenham(x4, y4, x1, y1)
points = p1 + p2 + p3 + p4
x = [p[0] for p in points]
y = [p[1] for p in points]
plt.scatter(x, y)
plt.grid(True)
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Rectangle using Bresenham's Algorithm")
plt.axis("equal")
plt.show()
```

22/9/25

Result:

Hence the output is verified

Scanned with CamScanner



Scanned with CamScanner

11/8

Draw a Traffic Signal

Aim:

To write a python program to draw a traffic signal using circles & also fill colors

Program:

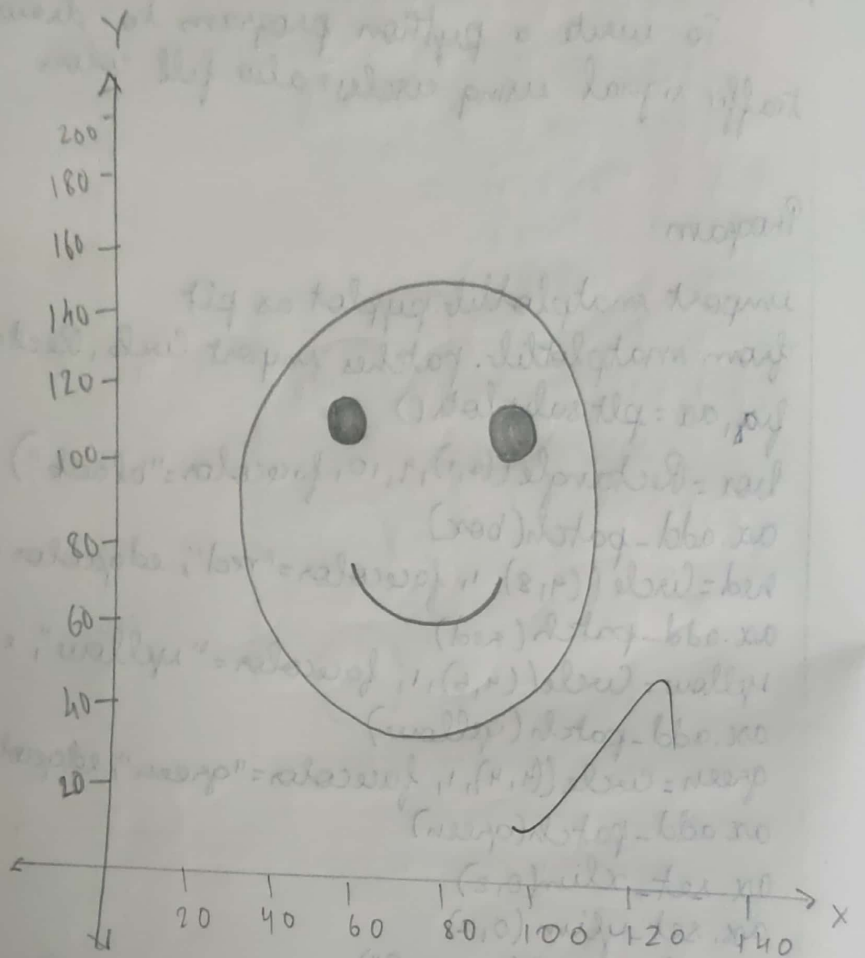
```
import matplotlib.pyplot as plt
from matplotlib.patches import Circle, Rectangle
fig, ax = plt.subplots()
box = Rectangle((2,1),4,10, facecolor="black")
ax.add_patch(box)
red = Circle((4,8),1, facecolor="red", edgecolor="white")
ax.add_patch(red)
yellow = Circle((4,6),1, facecolor="yellow", edgecolor="white")
ax.add_patch(yellow)
green = Circle((4,4),1, facecolor="green", edgecolor="white")
ax.add_patch(green)
ax.set_xlim(0,8)
ax.set_ylim(0,12)
ax.set_aspect("equal")
ax.grid(True)
plt.title("Traffic Signal")
plt.grid
plt.show()
```

22/9/20

Result:

Hence the output is verified

Scanned with CamScanner



Scanned with CamScanner

11/8

Draw a Smiling Face

Aim:

To write a python program to draw a smiling face using circles which includes eyes, nose and mouth.

Program:

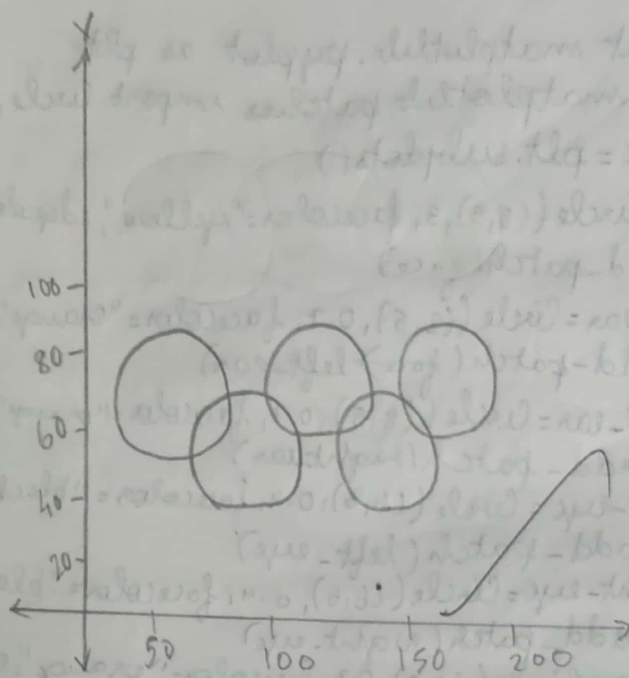
```
import matplotlib.pyplot as plt
from matplotlib.patches import Circle, Arc
fig, ax = plt.subplots()
face = Circle((5,5),3, facecolor="yellow", edgecolor="black")
ax.add_patch(face)
left_ear = Circle((2,5),0.7, facecolor="orange", edgecolor="black")
ax.add_patch(left_ear)
right_ear = Circle((8,5),0.7, facecolor="orange", edgecolor="black")
ax.add_patch(right_ear)
left_eye = Circle((4,6),0.4, facecolor="black")
ax.add_patch(left_eye)
right_eye = Circle((6,6),0.4, facecolor="black")
ax.add_patch(right_eye)
nose = Circle((5,5),0.3, facecolor="orange", edgecolor="black")
ax.add_patch(nose)
smile = Arc((5,4.5),3,2, angle=0, theta1=209, theta2=340,
        color="black", linewidth=3)
ax.add_patch(smile)
ax.set_xlim(0,10)
ax.set_ylim(1,9)
ax.set_aspect("equal")
ax.grid(True)
plt.title("Smiling face")
plt.show()
```

22/9/26

Result:

Hence, the output is verified

Scanned with CamScanner



Scanned with CamScanner

1.18

Draw Olympic Loop

Aim:

To write a python program to draw the olympic loop using ellipse

Program:

```
import matplotlib.pyplot as plt
from matplotlib.patches import Ellipse
fig, ax = plt.subplots()
ring1 = Ellipse((3,5), 2, 1.5, fill=False, edgecolor="blue",
linewidth=4)
ring2 = Ellipse((5,5), 2, 1.5, fill=False, edgecolor="black",
linewidth=4)
ring3 = Ellipse((7,5), 2, 1.5, fill=False, edgecolor="red",
linewidth=4)
ring4 = Ellipse((4,4), 2, 1.5, fill=False, edgecolor="yellow",
linewidth=4)
ring5 = Ellipse((6,4), 2, 1.5, fill=False, edgecolor="green",
linewidth=4)
ax.add_patch(ring1)
ax.add_patch(ring2)
ax.add_patch(ring3)
ax.add_patch(ring4)
ax.add_patch(ring5)
ax.set_xlim(1,9)
ax.set_ylim(2.5,6.5)
ax.set_aspect("equal")
ax.axis("off")
plt.title("Olympic Logo")
plt.show()
```

27/9/24

Result:

Hence, the output is verified

Scanned with CamScanner

11/8

Create a Butterfly using Ellipse

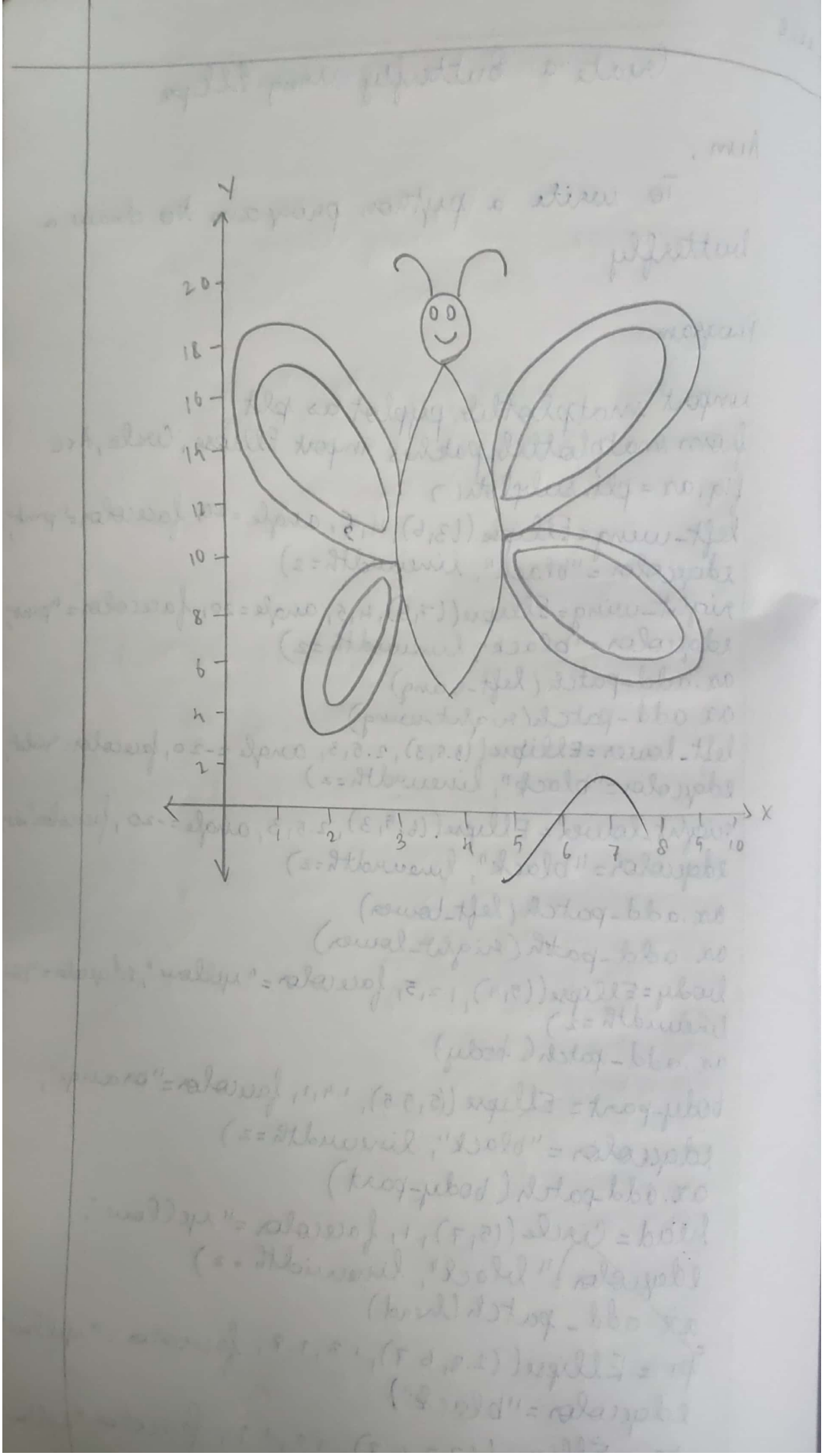
Aim:

To write a python program to draw a butterfly

Program:

```
import matplotlib.pyplot as plt
from matplotlib.patches import Ellipse, Circle, Arc
fig, ax = plt.subplots()
left_wing = Ellipse((3,6), 4, 5, angle=20, facecolor="pink",
edgecolor="black", linewidth=2)
right_wing = Ellipse((7,6), 4, 5, angle=20, facecolor="pink",
edgecolor="black", linewidth=2)
ax.add_patch(left_wing)
ax.add_patch(right_wing)
left_lower = Ellipse((3.5,3), 2.5, 3, angle=-20, facecolor="violet",
edgecolor="black", linewidth=2)
right_lower = Ellipse((6.5,3), 2.5, 3, angle=-20, facecolor="violet",
edgecolor="black", linewidth=2)
ax.add_patch(left_lower)
ax.add_path(right_lower)
body = Ellipse((5,4), 1.2, 3, facecolor="yellow", edgecolor="black",
linewidth=2)
ax.add_patch(body)
body_part = Ellipse((5,5.6), 1.4, 1, facecolor="orange",
edgecolor="black", linewidth=2)
ax.add_patch(body_part)
head = Circle((5,7), 1, facecolor="yellow",
edgecolor="black", linewidth=2)
ax.add_patch(head)
p1 = Ellipse((2.8,6.7), 1.3, 1.8, facecolor="yellow",
edgecolor="black")
p2 = Ellipse((7.2,6.7), 1.3, 1.8, facecolor="yellow",
edgecolor="black")
```

Scanned with CamScanner



Scanned with CamScanner

```
ax.plot([4.6,4.2,4.0],[7.8,8.5,8.7], color="black",
linewidth=2)
ax.plot([5.4,5.8,6.0],[7.8,8.5,8.7], color="black",
linewidth=2)
ax.add_patch(Circle((4.0,8.7),0.12, color="black"))
ax.add_patch(Circle((6.0,8.7),0.12, color="black"))
ax.set_xlim(0,10)
ax.set_ylim(0,10)
ax.set_aspect("equal")
ax.grid(True)
plt.title("Butterfly using Ellipse")
plt.show()
```

29/9/26

Result:

Hence, the output is verified

Scanned with CamScanner

8/9/26

To Implement Scan Conversion & Flood Fill

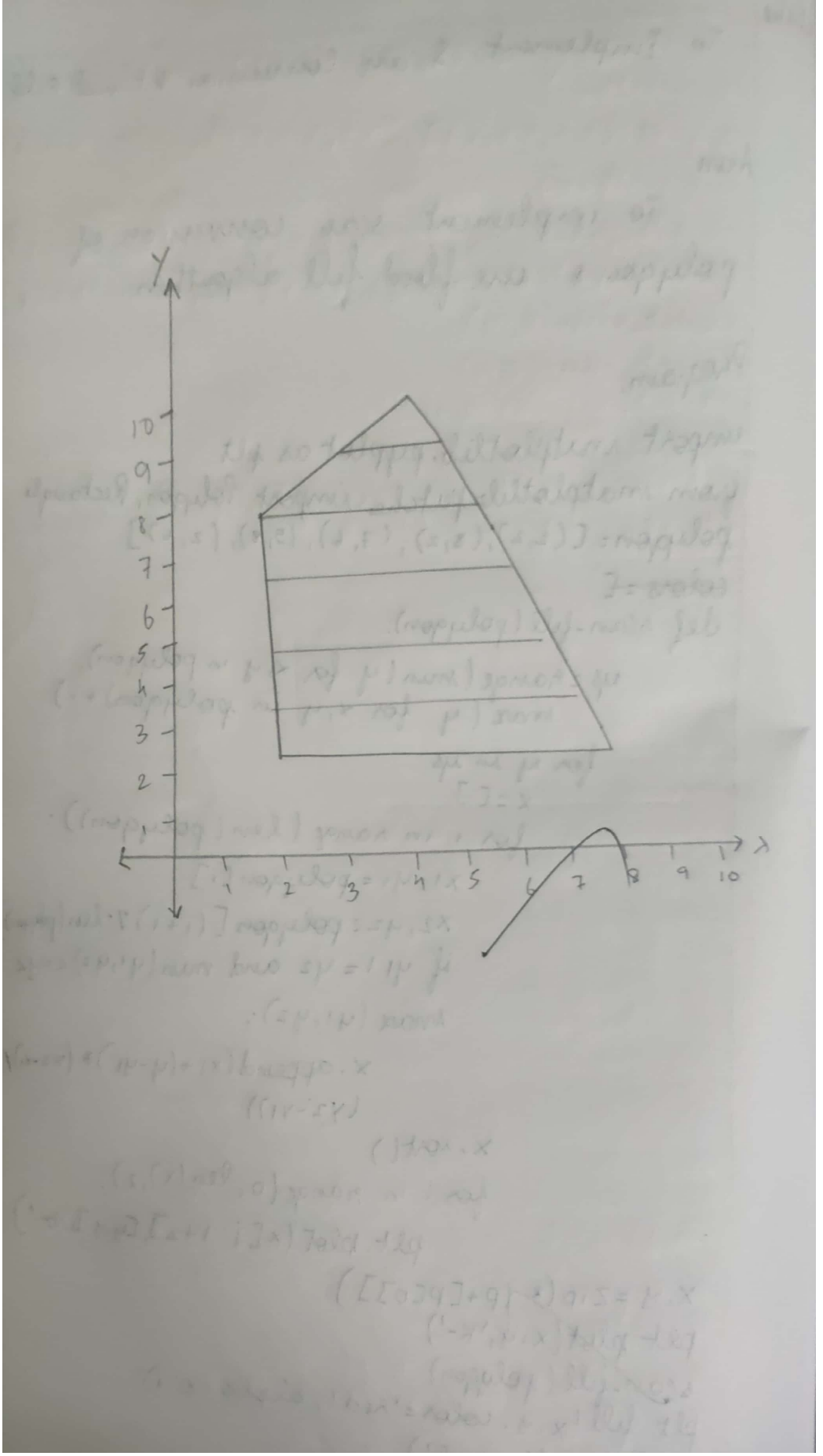
Aim:

To implement scan conversion of polygon & use flood fill algorithm

Program:

```
import matplotlib.pyplot as plt
from matplotlib.patches import Polygon, Rectangle
polygon = [(2,2),(8,2),(7,6),(5,8),(2,6)]
colors = [
def scan_fill(polygon):
    ys = range(min(y for x,y in polygon),
               max(y for x,y in polygon)+1)
    for y in ys:
        x = []
        for i in range(len(polygon)):
            x1,y1 = polygon[i]
            x2,y2 = polygon[(i+1)%len(polygon)]
            if y1 != y2 and min(y1,y2)<=y<
            max(y1,y2):
                x.append(x1+(y-y1)*(x2-x1)/
                (y2-y1))
        x.sort()
        for i in range(0,len(x),2):
            plt.plot(x[i:i+2],[y,y],'b')
x,y = zip(*polygon+[polygon[0]])
plt.plot(x,y,'k-')
scan_fill(polygon)
plt.fill(x,y,color='red',alpha=0.1)
plt.axis('equal')
```

Scanned with CamScanner



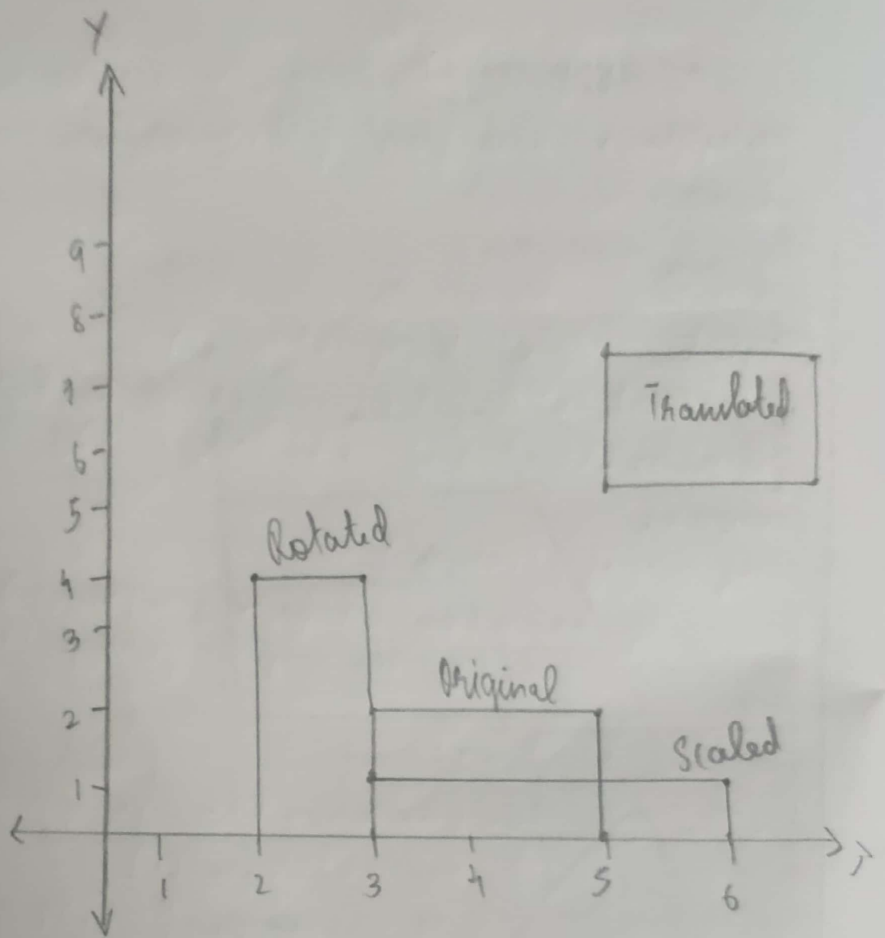
Scanned with CamScanner

```
plt.title("Scan conversion and flood fill")
plt.show()
```

Result:

Hence the output is verified successfully.

Scanned with CamScanner



Scanned with CamScanner

Implementation of 2D Shapes and To perform Basic Transformations

Aim:

To draw a 2D shape & perform basic transformation

Program:

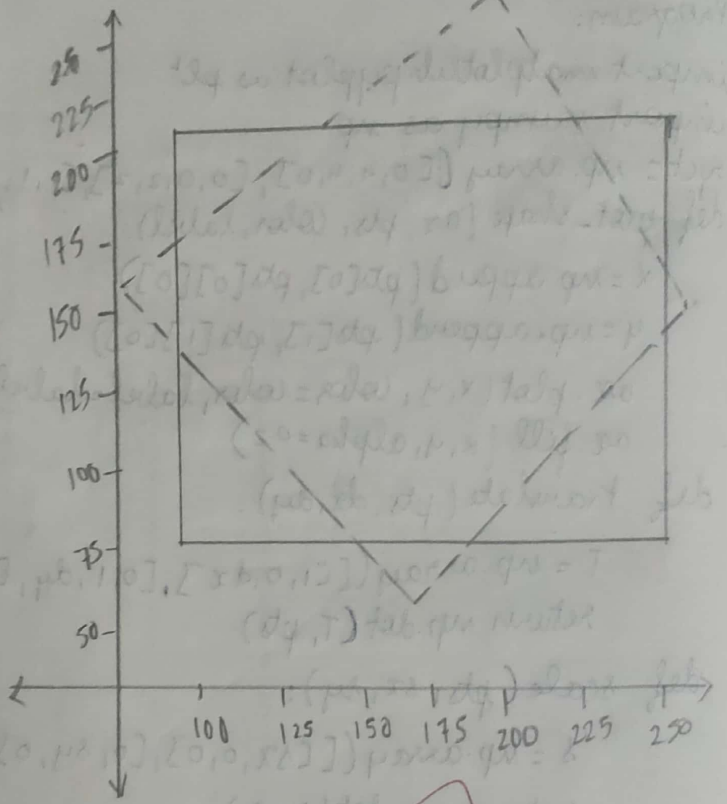
```
import matplotlib.pyplot as plt
import numpy as np
rect = np.array([[0,4,4,0],[0,0,2,2],[1,1,1,1]])
def plot_shape(ax, pts, color, label):
    x = np.append(pts[0], pts[0][0])
    y = np.append(pts[1], pts[1][0])
    ax.plot(x, y, color=color, label=label)
    ax.fill(x, y, alpha=0.2)
def translate(pts, dx, dy):
    T = np.array([[1,0,dx],[0,1,dy],[0,0,1]])
    return np.dot(T, pts)
def scale(pts, sx, sy):
    S = np.array([[sx,0,0],[0,sy,0],[0,0,1]])
    return np.dot(S, pts)
t_rect = translate(rect,3,4)
t_rect = scale(3.5,0.5)
t_rect = rotate(90)
plot_shape(ax, rect, "blue", "Original Rect")
plot_shape(ax, t_rect, "green", "Translated")
plot_shape(ax, s_rect, "orange", "Scaled")
plot_shape(ax, r_rect, "purple", "Rotated(90°)")
```

Result:

Hence the output is verified

Scanned with CamScanner

Polygon Clipping



Scanned with CamScanner

29.10.24

Polygon Clipping

Aim:

To implement polygon clipping using python programming.

Program:

```
pip install shapely matplotlib
import matplotlib.pyplot as plt
from shapely.geometry import Polygon
poly = Polygon([(100,150),(200,250),(300,150),(200,50)])
clip = Polygon([(120,180),(120,200),(280,200),(280,80)])
result = poly.intersection(clip)
x,y = poly.exterior.xy
plt.plot(x,y,'b--', label='Original')
x,y = clip.exterior.xy
plt.plot(x,y,'g--', label='Clip')
x,y = result.exterior.xy
plt.fill(x,y, alpha=.5, label='Clipped')
plt.title('Sutherland-Hodgman Clipping')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid()
plt.show()
print("Clipped Vertices:", list(result.exterior.
coords))
```

Result:

Therefore the output has been verified successfully.

Scanned with CamScanner

29.10.26

3D Transformations

Aim:

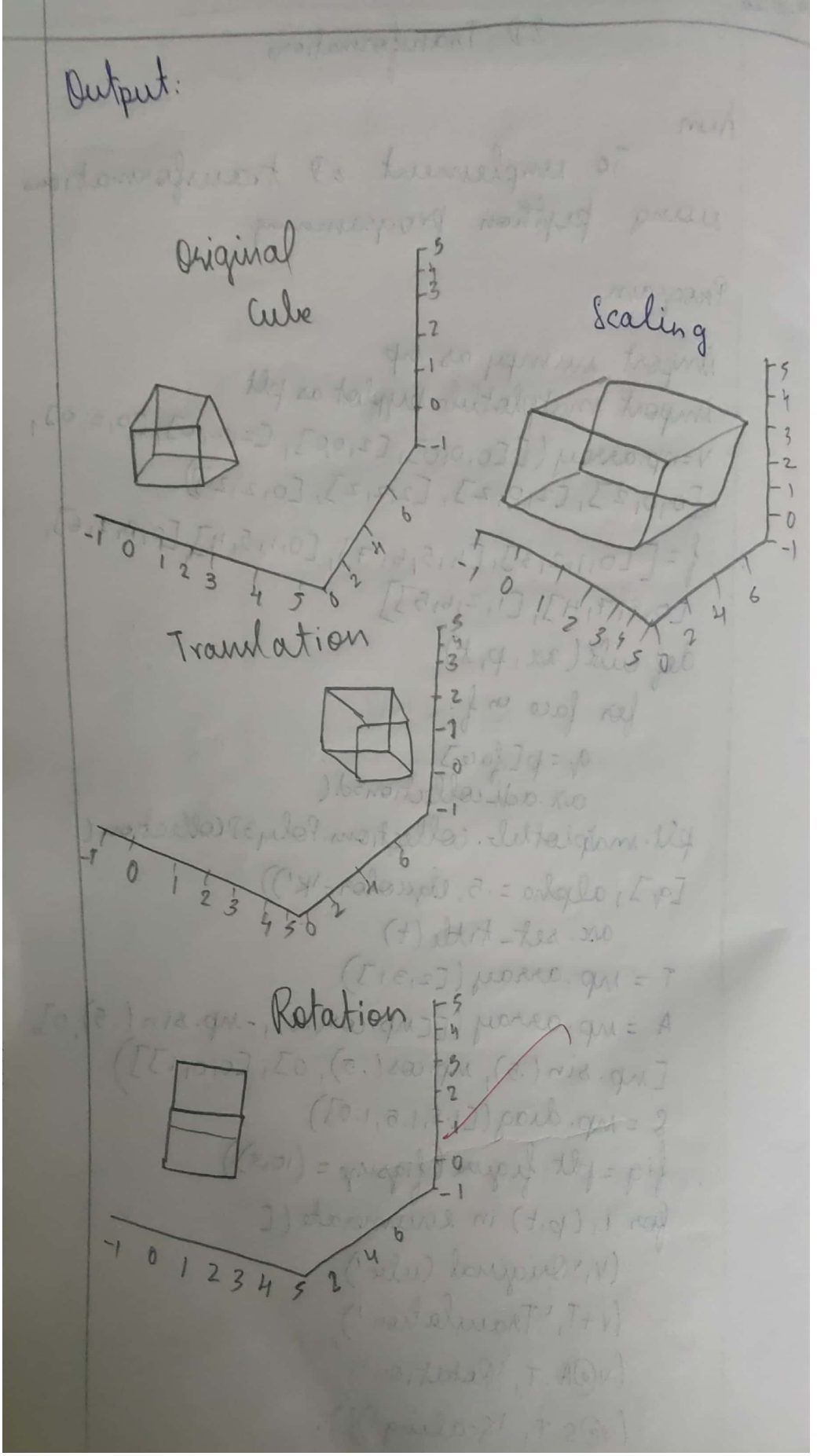
To implement 3D transformations using python programming

Program:

```
import numpy as np
import matplotlib.pyplot as plt
v=np.array([[0,0,0],[2,0,0],[2,2,0],[0,2,0],
[0,0,2],[2,0,2],[2,2,2],[0,2,2]])
f=[[0,1,2,3],[4,5,6,7],[0,1,5,4],[2,3,7,6],
[0,3,7,4],[1,2,6,5]]
def cube(ax,p,t):
    for face in f:
        q=p[face]
        ax.add_collection3d(
plt.matplotlib.collections.Poly3DCollection(
[q],alpha=.5,edgecolor='k'))
    ax.set_title(t)
T=np.array([2,3,1])
A=np.array([[np.cos(.5),-np.sin(.5),0],
[np.sin(.5),np.cos(.5),0],[0,0,1]])
S=np.diag([1.5,1.5,1.0])
fig=plt.figure(figsize=(10,3))
for i,(p,t) in enumerate([
    (v,'Original Cube'),
    (v+T,'Translation'),
    (v@A.T,'Rotation'),
    (v@S.T,'Scaling')]):
```

Scanned with CamScanner

Output:



Scanned with CamScanner

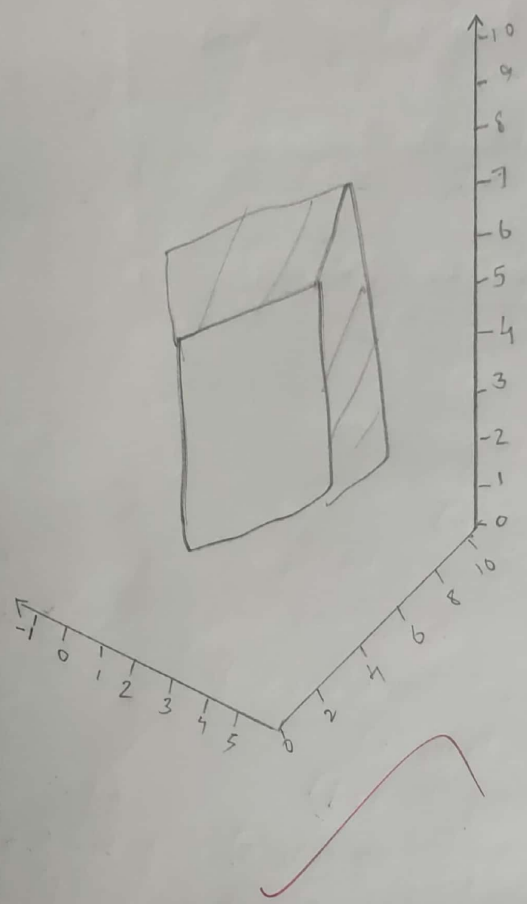
```
ax = fig.add_subplot(1,4,i+1, projection='3d')
Cube(ax,p,t)
plt.tight_layout()
plt.show()
```

Result:

Therefore the output has been verified.

Scanned with CamScanner

Output:



Scanned with CamScanner

3D Objects with Colour and Shading

Aim:

To create a 3D cube using polygon faces and apply basic colours and flat shading

Program:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import
Poly3DCollection

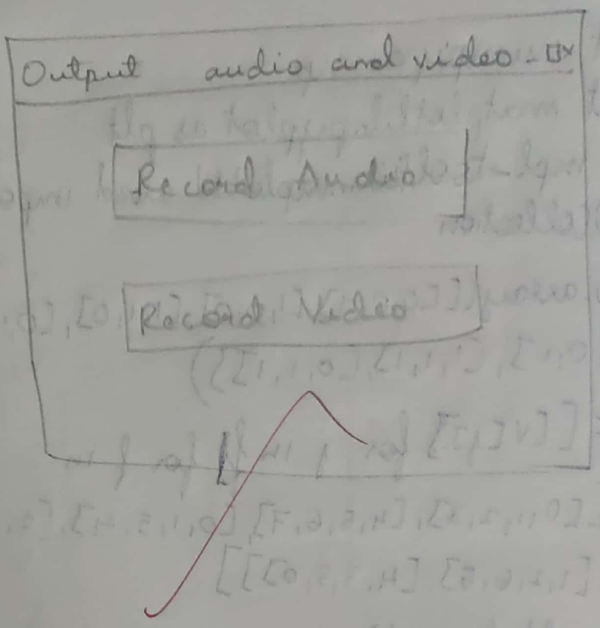
v = np.array([[0,0,0],[1,0,0],[1,1,0],[0,1,0],[0,0,1],
    [1,0,1],[1,1,1],[0,1,1]])
faces = [[v[j] for j in f] for f in
    [[0,1,2,3],[4,5,6,7],[0,1,5,4],[2,3,7,6],
    [1,2,6,5],[4,7,3,0]]]
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
obj = Poly3DCollection(faces,
facecolors=['red','blue','green','yellow','cyan',
'orange'], edgecolors='black')
ax.add_collection3d(obj)
ax.set_xlim(0,1)
ax.set_ylim(0,1)
ax.set_zlim(0,1)
plt.show()
```

Result:

Thus a 3D cube has been successfully rendered.

Scanned with CamScanner

Output :



Scanned with CamScanner

Multimedia Application

Aim:

To develop an application that displays images and plays audio and video

Program:

```
import tkinter as tk
from PIL import Image, ImageTk
from playsound import playsound
import cv2
import threading
def img():
    p = ImageTk.PhotoImage(Image.open('image.jpg'))
    l.config(image=p)
    l.image = p
def aud():
    playsound('audio.mp3')
def vid():
    c = cv2.VideoCapture('video.mp4')
    while c.isOpened():
        r, f = c.read()
        if not r: break
        cv2.imshow('video', f)
        if cv2.waitKey(25) == ord('q'):
            break
    c.release()
    cv2.destroyAllWindows()
w = tk.Tk()
l = tk.Label(w)
l.pack()
```

Scanned with CamScanner

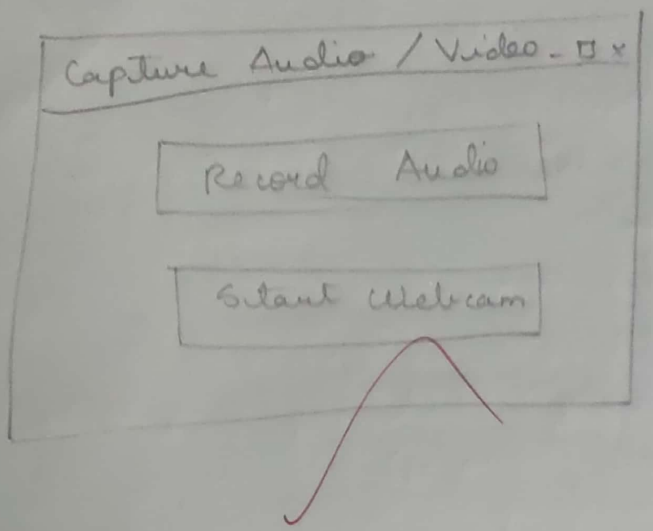
```
tk.Button(w, text='Image', command=img).pack()
tk.Button(w, text='Audio', command = lambda:
threading.Thread(target=aud).start()).pack()
tk.Button(w, text='Video', command=vid).pack()
w.mainloop()
```

Result:

Thus a multimedia application was created.

Scanned with CamScanner

Output :



Scanned with CamScanner

Capture Audio and Video

Aim:

To capture live video from a webcam and record audio from a microphone.

Program:

```
import cv2
import sounddevice as sd
from scipy.io.wavfile import write
import tkinter as tk
import threading
def audio():
    fs = 44100
    a = sd.rec(5*fs, samplerate = fs, channels=2)
    sd.wait()
    write('audio.wav', fs, a)
def video():
    c = cv2.VideoCapture(0)
    while True:
        r, f = c.read()
        if not r:
            break
        cv2.imshow('Webcam', f)
        if cv2.waitKey(1) == ord('q'):
            break
    c.release()
    cv2.destroyAllWindows()
w = tk.Tk()
tk.Button(w, text='Record Audio', command=
lambda: threading.Thread(target=audio).
start()).pack()
tk.Button(w, text='Start webcam',
command = lambda: threading.Thread(
target = video).start()).pack()
```

Scanned with CamScanner

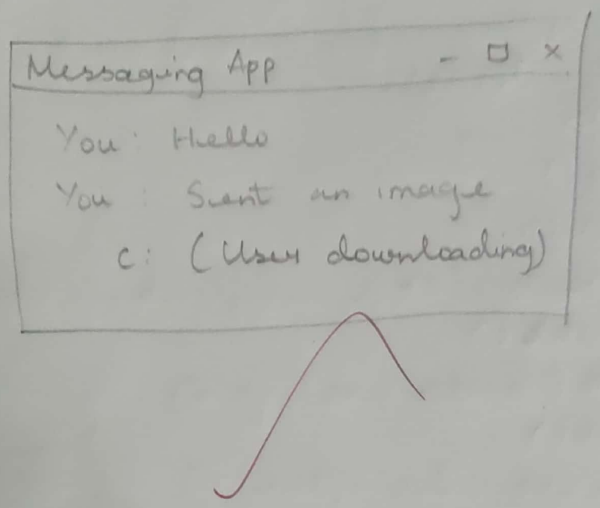
```
w.mainloop()
```

Result:

Thus, the audio was recorded and video was captured.

Scanned with CamScanner

Output



Scanned with CamScanner

Multimedia Messaging Application

Aim:

To create an application that supports text, image and audio messages.

Program:

```
import tkinter as tk
from tkinter import filedialog, scrolledtext
from PIL import Image, ImageTk
from playsound import playsound
import threading
def text():
    m = e.get()
    if m:
        c.insert(tk.END, 'you: '+m+'\n')
        e.delete(0, tk.END)
def img():
    p = filedialog.askopenfilename()
    if p:
        c.insert(tk.END, 'Image: '+p+'\n')
        i = ImageTk.PhotoImage(Image.
        open(p))
        l.config(img=i)
        l.img = i
def audio():
    p = filedialog.askopenfilename()
    if p:
        threading.Thread(target=lambda:
        playsound(p)).start()
        c.insert(tk.END, 'Audio: '+p+'\n')
```

Scanned with CamScanner

```
w = tk.Tk()
c = scrolledtext.ScrolledText(w, width=35,
height=10)
c.pack()
e = tk.Entry(w)
e.pack()
l = tk.Label(w)
l.pack()
tk.Button(w, text='Send Text', command=
text).pack()
tk.Button(w, text='Send Image', command=
ing).pack()
tk.Button(w, text='Send Audio',
command=audio).pack()
w.mainloop()
```

Result:

Thus, a GUI application was created.

Scanned with CamScanner

3D Modelling Using Blender

Aim:

To create a simple 3D model using Blender and apply:

- Basic Shading
- Textures

Procedure:

1. Launch Blender.
2. Model Creation:
   * Use add > mesh to insert basic shapes
   * Combine these objects to form a House.
   * Use Edit Mode for extruding
3. Shading:
   * Switch to the Shading Workspace
   * Add a new material
   * Adjust Base colour, Specular and Roughness
4. Texture Mapping:
   * Load an external image
   * Connect it to the Base color

Scanned with CamScanner

5. Lighting and Camera:

* Add light sources (sun)
* Position the camera

6. Render Settings:

* Choose Cycles or Eevee
* Adjust resolution and sampling
* Click Render > Render Image to product the output

Result:

A 3D object was succesfully modelled in Blender.

Scanned with CamScanner